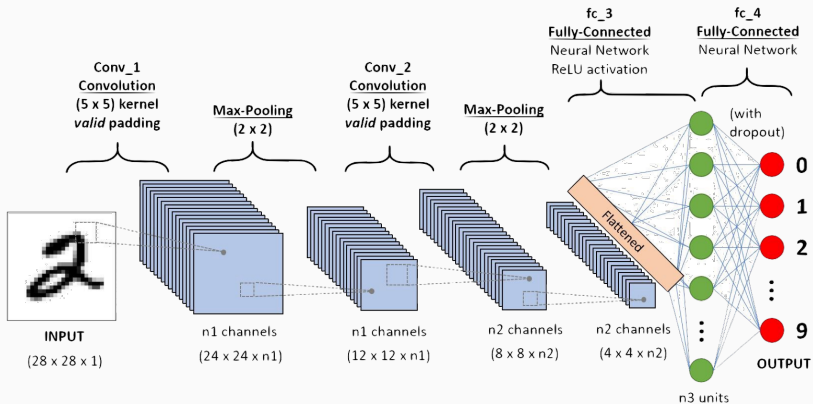


CS-GY 6923: Lecture 11

Training DNN, Graph Neural Networks

NYU Tandon School of Engineering, Akbar Rafiey

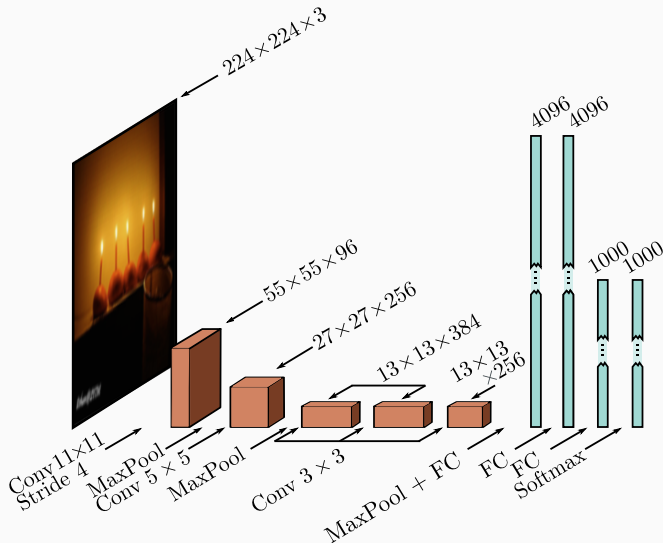
Overall network architecture



Each layer contains a 3D tensor of variables. Last few layers are standard fully connected layers.

AlexNet

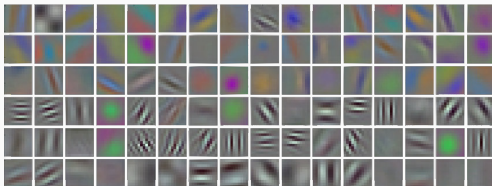
Almost all the 60 million parameters are in fully connected layers.



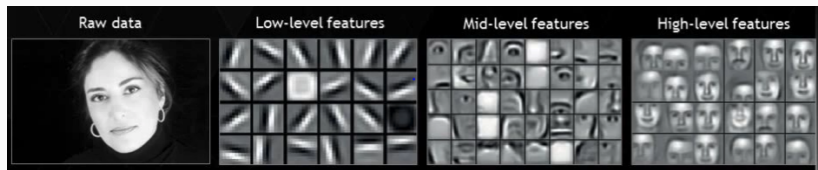
Understanding layers

What type of convolutional filters do we learn from gradient descent?

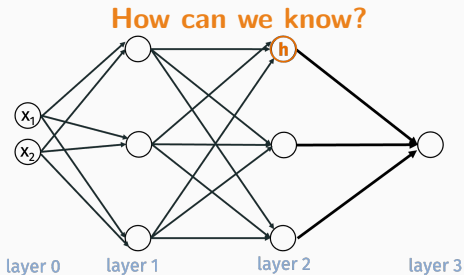
Lots of edge detectors in the first layer!



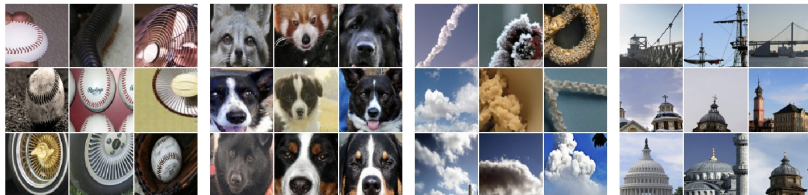
Other layers are harder to understand... but roughly hidden variables later in the network encode for “higher level features”:



Understanding layers

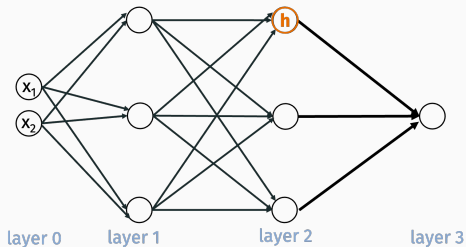


Go through dataset and find the inputs that most “excite” a given neuron h . I.e. for which $|h(\mathbf{x})|$ is largest.



Understanding layers

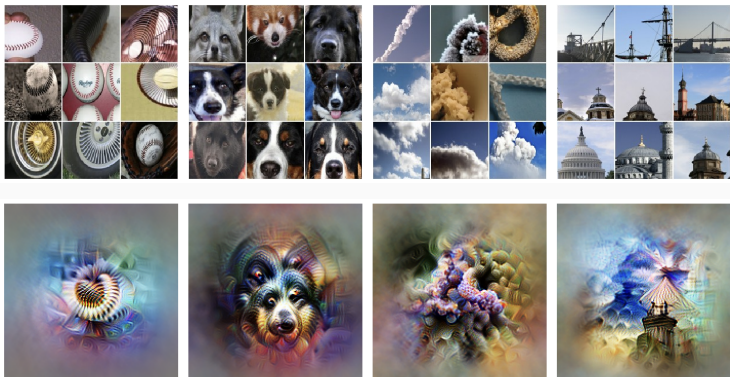
How can we know?



Alternative approach: Solve the optimization problem $\max_{\mathbf{x}} |h(\mathbf{x})|$ e.g. using gradient descent.

Understanding layers

There has been a lot of work on improving these methods by regularization. I.e. solve $\max_{\mathbf{x}} |h(\mathbf{x})| + g(\mathbf{x})$ where g constrains $\mathbf{x}$ to look more like a “natural image”.



If you are interested in learning more on these techniques, there is a great Distill article at: <https://distill.pub/2017/feature-visualization/>.

Understanding layers

Nodes at different layers have different layers capture increasingly more abstract concepts.



Parts (layers mixed4b & mixed4c)

Objects (layers mixed4d & mixed4e)

General obervation: Depth more important than width. Alexnet 2012 had 8 layers, modern convolutional nets can have 100s.

Tricks of the trade

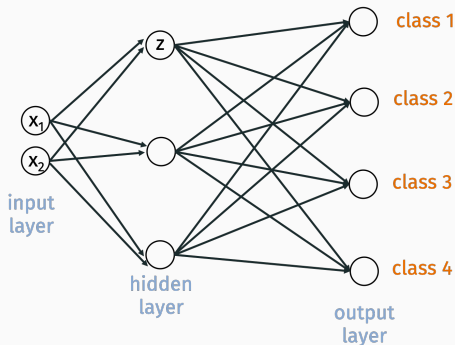
Beyond techniques discussed for general neural nets (back-prop, batch gradient descent, adaptive learning rates) training deep networks requires a lot of “tricks”.

- Batch normalization (accelerate training).
- Dropout (prevent over-fitting)
- Residual connections (accelerate training, allow for more depth – 100s of layers).
- Data augmentation.

And deep networks require lots of training data and lots of time.

Batch normalization

Start with any neural network architecture:



For input $\mathbf{x}$,

$$\bar{z} = \mathbf{w}^T \mathbf{x} + b$$

$$z = s(\bar{z})$$

where $\mathbf{w}$, b , and s are weights, bias, and non-linearity.

Batch normalization

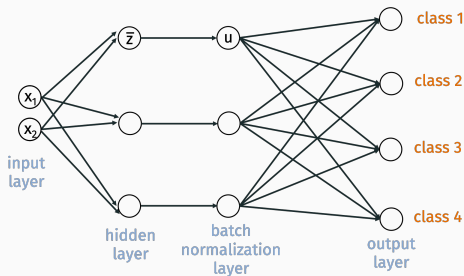
$\bar{z}$ is a function of the input $\mathbf{x}$. We can write it as $\bar{z}(\mathbf{x})$. Consider the mean and standard deviation of the hidden variable over a batch of data $\mathbf{x}_1 \dots, \mathbf{x}_m$:

$$\mu = \frac{1}{m} \sum_{j=1}^m \bar{z}(\mathbf{x}_j)$$
$$\sigma^2 = \frac{1}{m} \sum_{j=1}^n (\bar{z}(\mathbf{x}_j) - \mu)^2$$

Just as normalization (mean centering, scaling to unit variance) is sometimes used for input features, batch-norm applies normalization to learned features.

Batch normalization

Can add a batch normalization layer after any layer:



$$\bar{u} = \frac{\bar{z} - \mu}{\sigma}$$

$$u = s(\bar{u}).$$

Has the effect of mean-centering/normalizing $\bar{z}$. Typically we actually allow $u = s(\gamma \cdot \bar{u} + c)$ for learned parameters γ and c .

Batch normalization

Proposed in 2015: “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”, Ioffe, Szegedy.

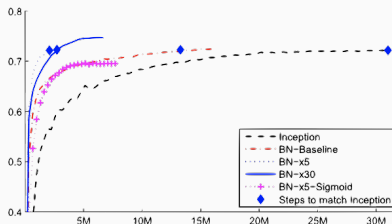


Figure 2: *Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.*

Doesn't change the expressive power of the network, but allows for significant convergence acceleration. (Inception is the CNN developed by Google that won ImageNet challenge in 2014.)

Advantages of BN

- Without BN: early layers might output very small or very large numbers
- Prevention of exploding or vanishing gradients
- With BN: activations are forced to have mean 0 and variance 1, gradients propagate through layers in a healthier range
- Lipschitzness of both the loss and the gradients are improved in models with BN.

How Does Batch Normalization Help Optimization?

Shibani Santurkar*
MIT
shibani@mit.edu

Dimitris Tsipras*
MIT
tsipras@mit.edu

Andrew Ilyas*
MIT
ailyas@mit.edu

Aleksander Madry
MIT
madry@mit.edu

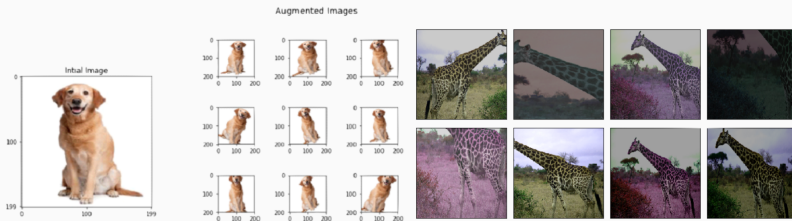
Abstract

Batch Normalization (BatchNorm) is a widely adopted technique that enables faster and more stable training of deep neural networks (DNNs). Despite its pervasiveness, the exact reasons for BatchNorm's effectiveness are still poorly understood. The popular belief is that this effectiveness stems from controlling the change of the layers' input distributions during training to reduce the so-called "internal covariate shift". In this work, we demonstrate that such distributional stability of layer inputs has little to do with the success of BatchNorm. Instead, we uncover a more fundamental impact of BatchNorm on the training process: it makes the optimization landscape significantly smoother. This smoothness induces a more predictive and stable behavior of the gradients, allowing for faster training.

Data augmentation

Great general tool to know about. **Main idea:**

- More training data typically leads to a more accurate model.
- Artificially enlarge training data with simple transformations.



Take training images and randomly shift, flip, rotate, skew, darken, lighten, shift colors, etc. to create new training images. **Final classifier will be more robust to these transformations.**

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Nitish Srivastava

Geoffrey Hinton

Alex Krizhevsky

Ilya Sutskever

Ruslan Salakhutdinov

Department of Computer Science

University of Toronto

10 Kings College Road, Rm 3302

Toronto, Ontario, M5S 3G4, Canada.

NITISH@CS.TORONTO.EDU

HINTON@CS.TORONTO.EDU

KRIZ@CS.TORONTO.EDU

ILYA@CS.TORONTO.EDU

RSALAKHU@CS.TORONTO.EDU

Editor: Yoshua Bengio

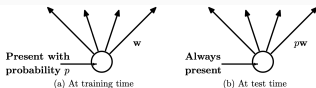


Figure 2: **Left:** A unit at training time that is present with probability p and is connected to units in the next layer with weights w . **Right:** At test time, the unit is always present and the weights are multiplied by p . The output at test time is same as the expected output at training time.

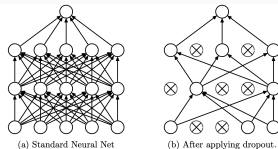


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

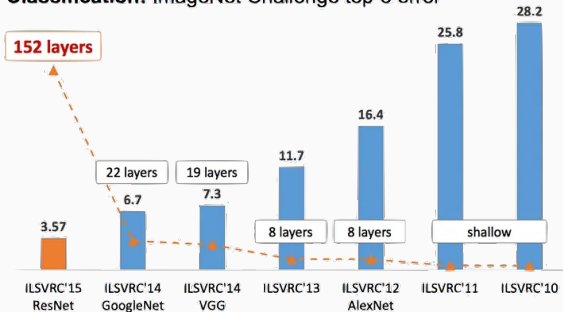
Why does it prevent overfitting?

Need to take a full course on neural networks/deep learning to learn more! State-of-the-art techniques are constantly evolving.

Deeper and deeper, bigger and bigger

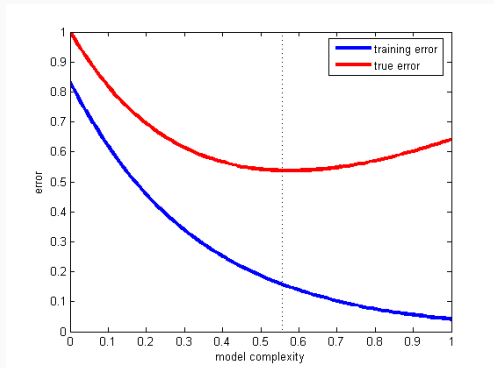
After AlexNet (8 layers, 60 million parameters) achieved state of the art performance on ImageNet, progress proceeded rapidly:

Classification: ImageNet Challenge top-5 error



Generalization for neural networks

Even with weight sharing, convolution, etc. modern neural networks typically have 100s of millions of parameters. And we don't train them with regularization. Intuitively we might expect them to overfit to training data.



Generalization for neural networks

In fact, we now know that modern neural nets can easily overfit to training data. This work showed that we can fit large vision data sets with random class labels to essentially perfect accuracy.

UNDERSTANDING DEEP LEARNING REQUIRES RE-THINKING GENERALIZATION

Chiyuan Zhang*

Massachusetts Institute of Technology
chiyuan@mit.edu

Samy Bengio

Google Brain
bengio@google.com

Moritz Hardt

Google Brain
mrtz@google.com

Benjamin Recht†

University of California, Berkeley
brecht@berkeley.edu

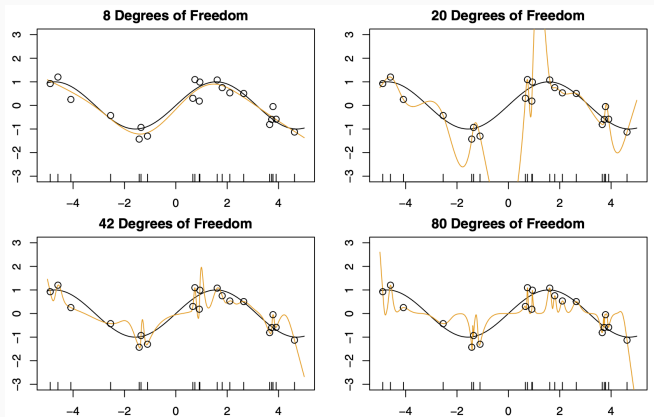
Oriol Vinyals

Google DeepMind
vinyals@google.com

But we don't always see a large gap between training and test error. **Don't take this to mean overfitting isn't a problem when using neural nets!** It's just not always a problem.

Generalization for neural networks

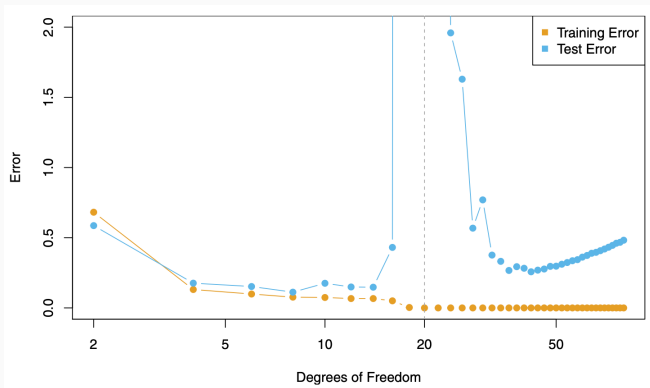
One growing realization is that this phenomena doesn't only apply to neural networks – it can also be true for fitting highly-overparameterized polynomials.



The choice of training algo (e.g. gradient descent) seems important.

Double descent

We sometimes see a “double descent curve” for these models. Test error is worst for “just barely” overparameterized models, but gets better with lots of overparameterization.



We don't usually see this same curve for neural networks.

The reason huge models work?



Andrew Gordon Wilson ✓ • 1st

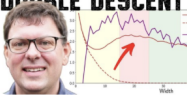
Professor at New York University

1mo • 🌐

...

My full interview with **Machine Learning Street Talk (MLST)** has just been posted! I really enjoyed this conversation. We talk about the bitter lesson, scientific discovery, Bayesian inference, mysterious phenomena, and key principles for building intelligent systems. <https://lnkd.in/d4nuDa3C>

DOUBLE DESCENT



The Real Reason Huge AI Models Actually Work

[youtube.com](https://www.youtube.com/watch?v=d4nuDa3C)

Overfitting in neural nets

Take away: Modern neural network overfit, but still seem fairly robust. Perform well on any new test data we throw at them.

Or do they?

Intriguing properties of neural networks

Christian Szegedy
Google Inc.

Wojciech Zaremba
New York University

Ilya Sutskever
Google Inc.

Joan Bruna
New York University

Dumitru Erhan
Google Inc.

Ian Goodfellow
University of Montreal

Rob Fergus
New York University
Facebook Inc.

Adversarial examples

Adversarial examples

Main discovery: It is possible to find imperceptibly small perturbations of input images that will fool deep neural networks. This seems to be a universal phenomenon.



Important: Random perturbations do not work!

Adversarial examples

How to find “good” perturbations:

Fix model f_{θ} , input $\mathbf{x}$, correct label y . Consider the loss $\ell(\theta, \mathbf{x}, y)$.

Solve the optimization problem:

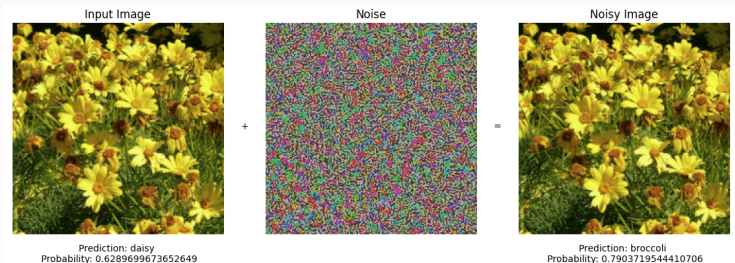
$$\max_{\delta, \|\delta\| \leq \epsilon} \ell(\theta, \mathbf{x} + \delta, y)$$

Can be solved using gradient descent! We just need to compute the derivative of the loss with respect to the image pixels.

Backprop can do this easily.

Adversarial examples

You can find your own adversarial examples for a model called Resnet18. The entire model + weights are available through PyTorch, so we do not need to train it ourselves (i.e. this is a pre-trained model).



Transfer Learning

State-of-the-art supervised learning models like neural networks learn **very good features**.

But they require lots and lots of data. ImageNet has 14 million labeled images. Mostly of everyday objects.

One-shot learning

What if you want to apply deep convolutional networks to a problem where you do not have a lot of **labeled data** in the first place?



quaffle



bludger



snitch

Example: Classify images of different Quidditch balls.

One-shot learning

Real example: Classify images of insects for use in agricultural applications in new localities.

Zero-Shot Insect Detection via Weak Language Supervision

**Benjamin Feuer,¹ Ameya Joshi,¹ Minsu Cho,¹ Kewal Jani,¹ Shivani Chiranjeevi,² Zi Kang Deng,³
Aditya Balu,² Asheesh K. Singh,² Soumik Sarkar,² Nirav Merchant,³ Arti Singh,²
Baskar Ganapathysubramanian,² Chinmay Hegde¹**

¹ New York University

² Iowa State University

³ University of Arizona

Aedes Vexans



Cretonotos Gangis



Daphnis Neril



Hypena Deceptalis

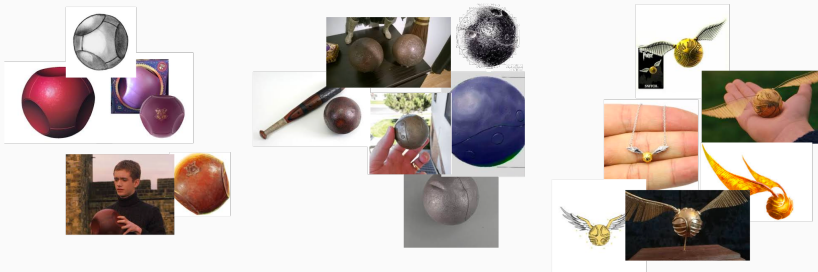


Pyralis Farinalis



One-shot learning

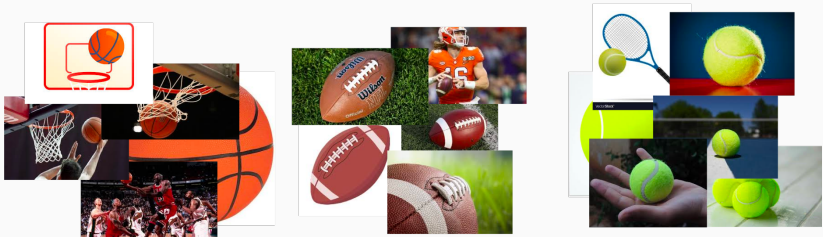
A human could probably achieve near perfect classification accuracy even given access to a **single labeled example** from each class:



Major question in ML: How? Can we design ML algorithms which can do the same?

Transfer learning

Transfer knowledge from one task we already know how to solve to another.



For example, we have learned from past experience that balls used in sports have consistent shapes, colors, and sizes. These features can be used to distinguish balls of different type.

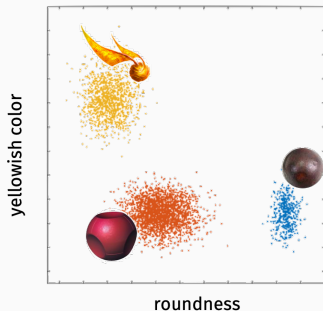
Feature learning

Examples of possible high-level features a human would learn:

		Classes					
							
Features	roundness	1	.1	1	.6	1	.4
	size relative to human hand	10	7	2	7	5	1
	yellowish color	.2	.1	1	.1	0	.9

Feature learning

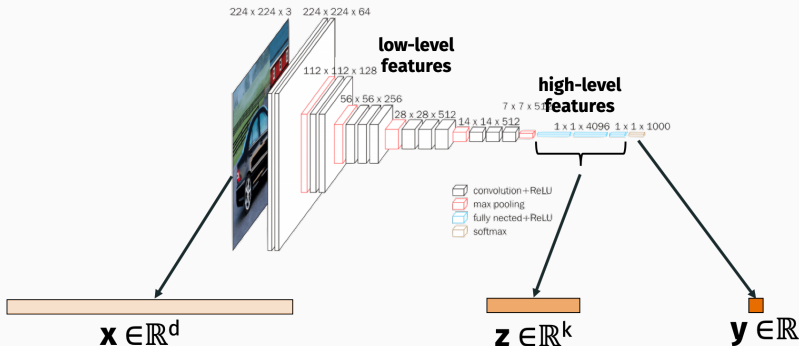
If these features are highly informative (i.e. lead to highly separable data) few training examples are needed to learn.



Might suffice to classify ball using nearest training example in feature space, even if just a handful of training examples.

Transfer learning

Empirical observation: Features learned when training models like deep neural nets seem to capture exactly these sorts of high-level properties.



Even if we can't put into words what each feature in $\mathbf{z}$ means...

Transfer learning

This is now a common technique in computer vision:

1. Download network trained on large image classification dataset (e.g. Imagenet).
2. Extract features $\mathbf{z}$ for any new image $\mathbf{x}$ by running it through the network up until layer before last.
3. Use these features in a simpler machine learning algorithm that requires less data (nearest neighbor, logistic regression, etc.).

This approach has even been used on the quidditch problem:

github.com/thatbrguy/Object-Detection-Quidditch

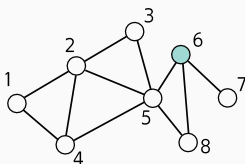
Graph Neural Networks (GNNs)

What is a graph?

A graph consists of a set of nodes or vertices, where pairs of nodes are connected by edges or links.

- $G = (V, E)$
- V is the set of nodes also called vertices
- $E \subseteq V \times V$ is the set of edges.
- Representation: adjacency matrix, dictionary, ...

a)



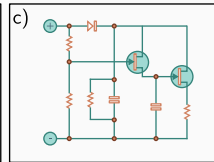
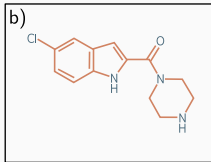
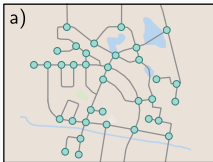
b)

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Graphs are everywhere

Some objects naturally take the form of graphs:

- Road networks: nodes are locations; edges are roads.
- Chemical molecules: nodes are atoms; edges are bonds.
- Electrical circuits: nodes are components/junctions; edges are electrical connections.

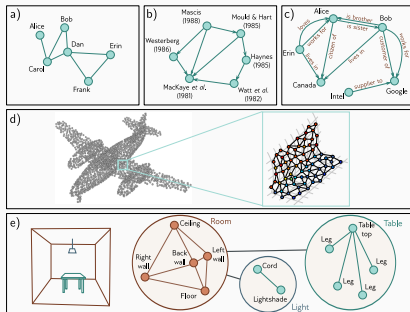


Graphs are everywhere

- Social networks: nodes are people; edges represent friendships.
- Wikipedia: nodes are articles; edges are hyperlinks.
- Point clouds: nodes are points; edges connect nearby points.
- Protein interactions: nodes are proteins; edges indicate interactions.

Types of graphs

- Undirected vs. directed edges (e.g., friendships vs. citations).
- Knowledge graphs
- Geometric graphs: built from point sets via K -nearest neighbors.
- Hierarchical graphs: component graphs (e.g., table, light, room) forming nodes in a larger graph.



Node and edge embeddings

Information is typically associated with nodes (node embeddings) and sometimes edges (edge embeddings).

A graph with n nodes and m edges can be encoded by three matrices:

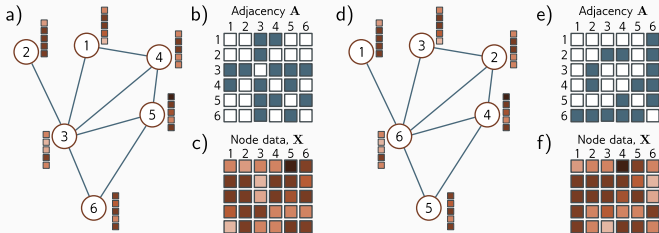
- Adjacency matrix $A \in \mathbb{R}^{n \times n}$ (graph structure).
- Node data matrix $X \in \mathbb{R}^{d \times n}$ (node embeddings).
- Edge data matrix $m \in \mathbb{R}^{d_E \times m}$ (edge embeddings).

For undirected graphs, A is symmetric; large sparse graphs can store connections as a list (m, n) .

Aside: permutation of node indices

Node indexing is arbitrary. Permuting node indices permutes columns of X and both rows/columns of A , but the underlying graph is unchanged.

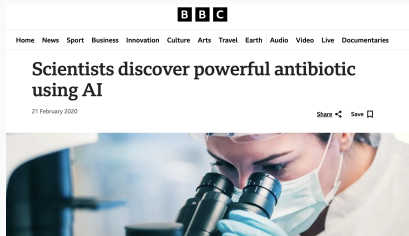
Any processing should be indifferent to such permutations.



Application 1: molecule classification

Task: predict whether the molecule is a potent drug.

- Do binary classification on whether the drug will inhibit certain bacteria (E.coli)

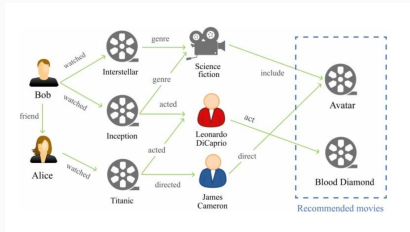


Application 2: recommender systems

A common task in social networks is recommendation.

Task: Based on user's preferences suggest new content.

Task: Using the existing links predict new links.



Graph Convolutional Neural Networks for Web-Scale Recommender Systems

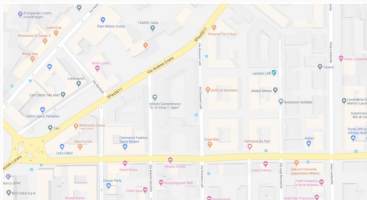
Rex Ying^{†*}, Ruining He^{*}, Kaifeng Chen^{†*}, Pong Eksombatchai^{*},

William L. Hamilton[†], Jure Leskovec^{†*}

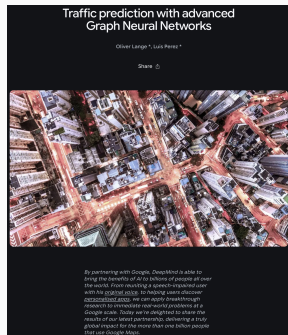
^{*}Pinterest, [†]Stanford University
{rex, kaifengchen, pong}@pinterest.com, {reying, wlei, jure}@stanford.edu

Application 3: ETA prediction

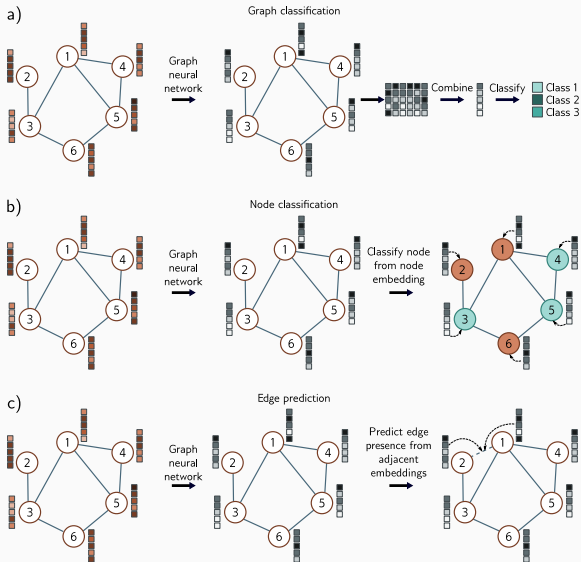
Transportation maps (e.g., Google Maps) modeled as graphs.



Task: Based on node features (speed, length, speed history,...) predict estimated arrival time.



Tasks: graph, node, edge



A Simple GNN

- Let $G(V, E)$ be an undirected graph where each node $v \in V$ has an associate feature vector $\mathbf{x}_v \in \mathbb{R}^d$ i.e. an embedding.
- Our goal is to update/improve the embedding of each node so that they can help with the final task, for example graph classification.

Question: How can we update the current embedding of v using structure of the graph and other nodes embeddings?

A Simple GNN

- Let $G(V, E)$ be an undirected graph where each node $v \in V$ has an associate feature vector $\mathbf{x}_v \in \mathbb{R}^d$ i.e. an embedding.
- Our goal is to update/improve the embedding of each node so that they can help with the final task, for example graph classification.
- A natural thing to do is to collect information from the neighbors of v . Let $N(v)$ be the set of nodes adjacent to v .
- This, in some sense, is similar to convolution we have seen for CNN, and preserves spatial properties.

A Simple Message Passing GNN (MPGNN)

Iterative message passing framework:

- At time $t = 0$, we have $h_v^{(0)} = \mathbf{x}_v$.
- At each iteration, every node collects messages from its neighbors and updates its own embedding
- Let $h_v^{(t)}$ be hidden state (i.e., embedding) of node v at step t :

$$a_v^{(t)} = \text{AGGREGATE}^{(t)}(\{h_u^{(t-1)} : u \in N(v)\})$$

$$h_v^{(t)} = \text{COMBINE}^{(t)}(h_v^{(t)}, a_v^{(t)})$$

What properties should AGGREGATE have? How about COMBINE?

A Simple Message Passing GNN (MPGNN)

- Let $h_v^{(t)}$ be hidden state (i.e., embedding) of node v at step t :

$$a_v^{(t)} = \text{AGGREGATE}^{(t)}(\{h_u^{(t-1)} : u \in N(v)\})$$

$$h_v^{(t)} = \text{COMBINE}^{(t)}(h_v^{(t)}, a_v^{(t)})$$

AGGREGATE must be a permutation-invariant function.

COMBINE better have nonlinearity.

Different choices of AGGREGATE and COMBINE give rise to different GNNs.

Examples of different GNNs

Graph Convolutional Networks (GCN) (Kipf and Welling, 2017):

- Neighborhood aggregation at node v (iteration t):

$$a_v^{(t)} = \sum_{u \in N(v)} h_u^{(t-1)}.$$

$$h_v^{(t)} = \sigma \left(b_t + W_t h_v^{(t-1)} + W_t a_v^{(t)} \right).$$

- In matrix form ($\mathbf{H}_t \in \mathbb{R}^{d \times n}$):

$$\mathbf{H}_t = \sigma \left(\mathbf{b}_t \mathbf{1}^\top + W_t \mathbf{H}_{t-1} (\mathbf{A} + \mathbf{I}) \right).$$

- There are several mean pooling variants where we normalize $a_v^{(t)}$ by $|N(v)|$ or $|N(v)| \cdot |N(u)|$ etc...

Examples of different GNNs

GraphSAGE (W Hamilton, Z Ying, J Leskovec, 2017):

$$a_v^{(t)} = \text{MAX} \left(\left\{ \text{ReLU}(W \cdot h_u^{(t-1)}) : u \in N(v) \right\} \right)$$
$$h_v^{(t)} = \sigma(W^{(t)} \cdot \text{CONCAT}(h_v^{(t)}, a_v^{(t)}))$$

The function applied before the max pooling can be an arbitrarily deep MLP.

(There are lots of other popular GNNs, for example Graph Attention Networks (GAT))

Depending on the task, whether it is graph classification, node classification, graph regression, etc., we can design a READOUT function that takes in (i.e., aggregates) all the embeddings of nodes and returns an appropriate output.

$$h_G = \text{READOUT}(\{h_v^{(k)} : v \in V\})$$

Toxic vs. harmless molecules (graph classification)

Inputs: $\mathbf{A} \in \mathbb{R}^{n \times n}$ (adjacency of the graph), $\mathbf{X} \in \mathbb{R}^{n \times d}$.

$$\mathbf{H}_1 = \sigma\left(\mathbf{b}_0 \mathbf{1}^\top + W_0 \mathbf{X}(\mathbf{A} + \mathbf{I})\right),$$

...

$$\mathbf{H}_K = \sigma\left(\mathbf{b}_{K-1} \mathbf{1}^\top + W_{K-1} \mathbf{H}_{K-1}(\mathbf{A} + \mathbf{I})\right),$$

$$f[\mathbf{X}, \mathbf{A}, \theta] = \text{sig}\left[\mathbf{b}_K + \omega_K \mathbf{H}_K \frac{\mathbf{1}}{n}\right].$$

Training: SGD with binary cross-entropy.

Check out these nice blog posts on GNNs:

- <https://mbernste.github.io/posts/gcn/>
- Very nice blog with a GNN playground at the end
<https://distill.pub/2021/gnn-intro/>